

MACARON: Multi-cloud region Aware Cache Auto-configuRatiON

Design Document

Fulun Ma, Hojin Park

20 Mar 2024

Version: 1.2

Changelog

Version	Date	Changelog
V1.0	2024.3.18	Added draft of introduction, design considerations and system architecture
V1.1	2024.3.19	Added draft of data design, implementation overview and design models
V1.2	2024.3.20	Added draft of rest sections

Introduction

This document serves as a comprehensive guide to the architectural design choices for the MACARON [1] project, aimed at enhancing data management across multi-cloud environments. The core objective of this project is to address the inefficiencies in data synchronization and access within distributed computing landscapes, specifically focusing on minimizing data egress costs and improving performance in cloud-based applications. By exploring innovative caching strategies and synchronization mechanisms, the project seeks to provide scalable, cost-effective solutions for a wide range of users, from cloud service administrators to data scientists.

The document outlines two main design decisions: the Immediate Notification System with On-demand Synchronization, and the Central Metadata Registry with On-demand Synchronization. Both architectures propose novel approaches to managing data synchronization between local caches and a central data lake, leveraging Amazon Web Services (AWS) for efficient, real-time data handling. The first design focuses on using placeholder objects and AWS Lambda for dynamic data fetching, whereas the second design employs Amazon DynamoDB to track and synchronize data objects systematically. These designs were conceived to mitigate the challenges of data management in multi-cloud settings.

Design Considerations

Assumptions

- **Data Variability and Volume:** The project assumes a high variability and volume of data across different cloud environments, necessitating robust and flexible synchronization mechanisms.

- **Network Reliability:** It is presumed that network connectivity between local caches and the central data lake is reliable, although not immune to occasional disruptions.
- **User Interaction:** Users are expected to interact with the system via predefined interfaces, without the need for deep technical knowledge of the underlying architecture.
- **High Egress Data Costs:** A fundamental assumption of the project is the high cost associated with data egress across cloud platforms. This is a critical factor influencing the design of the caching and synchronization mechanisms, aiming to minimize these costs by optimizing when and how data is transferred between local caches and the central data lake.

Constraints

- **Budgetary Limitations:** Financial resources are finite, impacting the choice of cloud services and the scale at which the system can be deployed.
- **Technological Boundaries:** The project is limited to the use of AWS services, including S3, Lambda, DynamoDB, and API Gateway, constraining the design to the capabilities and limitations of these tools.
- **Time Frame:** The project must be completed within a set timeline, necessitating efficient development and deployment strategies.

System Environment

The MACARON project is exclusively designed to operate within the AWS cloud environment. This setup involves utilizing AWS services for both the local caches and the central data lake, ensuring a seamless and integrated infrastructure. The deployment on AWS facilitates a uniform platform for all components of the project, leveraging AWS's global infrastructure for reliability and scalability. This environment requires careful planning around AWS service usage, data transfer methods, and the management of egress costs associated with data movements within the AWS ecosystem.

Design Methodology

The project adopts an iterative design approach, enabling the gradual refinement of the system based on feedback and testing. This methodology consists of initial design and implementation, followed by cycles of testing, evaluation, and adjustments. The system's architecture is modular, allowing for independent development and integration of components, facilitating scalability and adaptability to changing requirements.

System Architecture

Immediate Notification System with On-demand Synchronization

The architecture for the Immediate Notification System with On-demand Synchronization is designed to ensure efficient data handling and responsiveness in a multi-cloud environment. The diagram illustrates a client-centric approach where multiple clients interact with their respective local caches and a central data lake.

Central Metadata Registry with On-demand Synchronization

The architecture for the Central Metadata Registry with On-demand Synchronization is designed to streamline the coordination between distributed caches and a unified data lake. The included diagram highlights the interplay between clients, caching layers, and a metadata tracking system. Here's a detailed overview of the architecture:

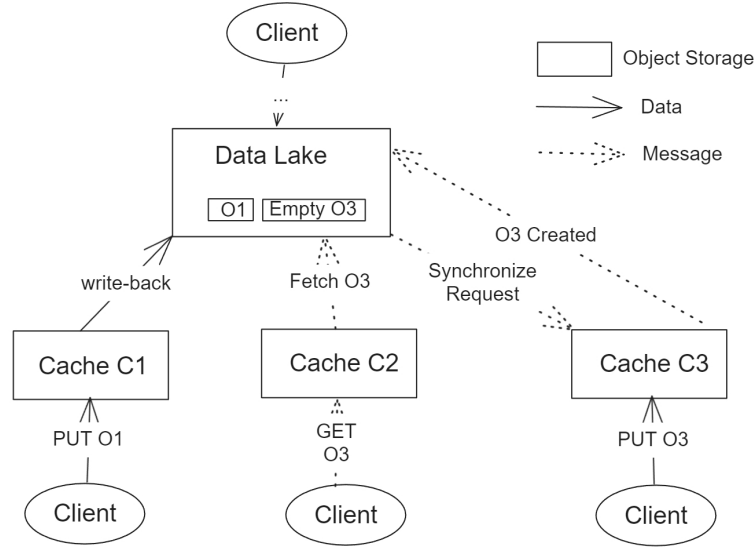


Figure 1: Architecture of Immediate Notification System with On-demand Synchronization

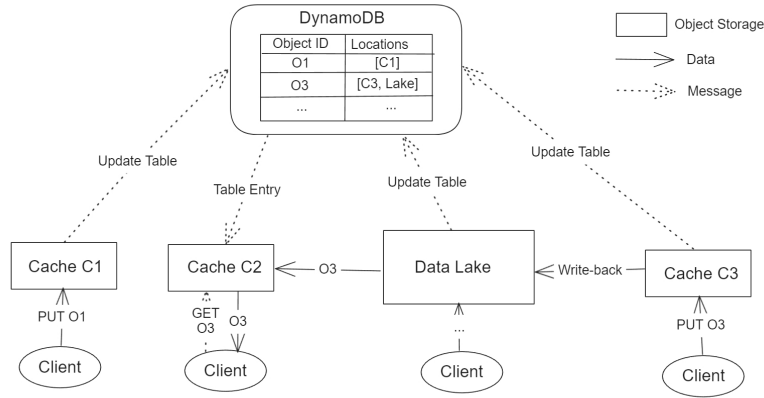


Figure 2: Architecture of Central Metadata Registry with On-demand Synchronization

Data Design

Data Descriptions and Formats

- **Data Objects:** These are the discrete units of data managed by the system. They can range from small files to large blobs of unstructured data.
- **Format:** Data objects are stored in a binary format, with metadata in JSON to encapsulate attributes like the object ID, size, last modified timestamp, owner, and access permissions.
- **Metadata:** For each data object, a corresponding metadata entry is created that includes keys and values representing object attributes

Data Structure

Metadata Registry utilizes a DynamoDB table with the following schema:

- ObjectID: String – Primary Key.
- Size: Number – Size of the object in bytes.
- LastModified: String – ISO8601 timestamp of the last modification.
- Owner: String – Identifier for the owner of the object.
- Permissions: String – Access permissions for the object.
- IsSynchronized: Boolean – Flag indicating synchronization status.
- Locations: List – Caches and/or data lake where the object is stored.

Data Flow

The data flow in the MACARON project can be represented by two main diagrams corresponding to the two system architectures.

Implementation Overview

See the diagram titled "Architecture of MACARON Core".

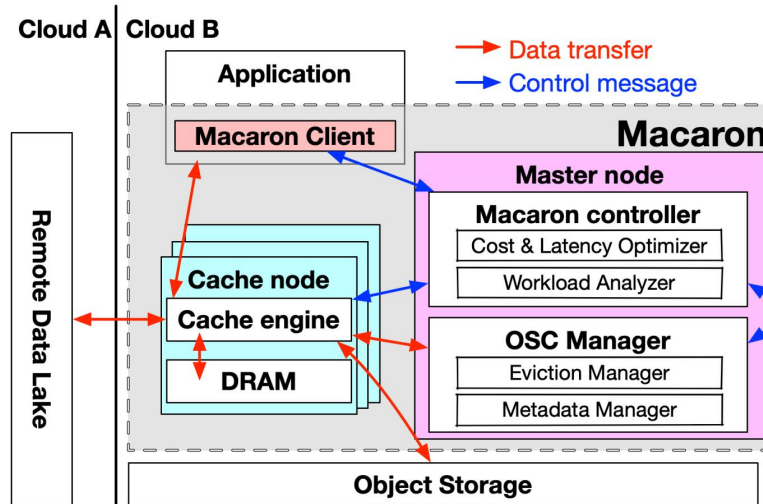


Figure 3: Architecture of MACARON Core

Design Models

The system's object model is intricately structured, comprising Cache Node Objects that encapsulate both the physical data and its associated metadata, ensuring a seamless data retrieval process. These objects are managed by Master Node Objects, which are equipped with optimization policies and workload profiles that guide the system's caching strategies and data synchronization protocols. This detailed object model is critical for understanding how data is handled within the cache nodes and how it is orchestrated by the master node to optimize for performance, cost, and latency.

Expanding further, the data flow model delineates the journey of data as it traverses through the system—from ingestion to eviction. This model emphasizes the data's lifecycle, highlighting the synchronization processes that reconcile data states between local caches and the remote data lake. External services and data integration are mapped within this model to show how cloud services like AWS S3, Lambda, and DynamoDB interlink to maintain data consistency and system responsiveness. Furthermore, the integration with monitoring and optimization tools showcases the

project's commitment to continuous improvement and adaptability. Together, these design modules underscore the project's responsiveness to the complex needs of multi-cloud data management, revealing the MACARON project's robust, scalable, and efficient architecture.

Test Design

The Test Design will involve simulated workload tests to gauge the efficiency of cache operations, data synchronization accuracy, and the responsiveness of the eviction policies under differing data volumes and access patterns. Key metrics such as cache hit/miss ratios, data retrieval latency, synchronization times, and egress cost reductions will be measured and compared against baseline figures to assess improvements. Systematic fault injection will test the robustness of error handling and recovery procedures. The results from these tests will be collated and displayed in a result table, encompassing a suite of scenarios to provide a clear, quantified view of the system's behavior.

Deployment Model

The deployment model for the MACARON project is designed to leverage cloud-native services, facilitating a smooth transition from development to production within the AWS ecosystem. The strategy includes containerization of the MACARON components for ease of orchestration with services like AWS Elastic Container Service (ECS) or Kubernetes via Amazon EKS, enabling scalable and resilient deployment across multiple availability zones. Continuous integration and continuous deployment (CI/CD) pipelines will be established using AWS CodePipeline and CodeBuild, allowing for automated testing and deployment phases that ensure code changes are seamlessly integrated and delivered to production environments.

Risk/Challenges

Tools & Dependencies

Terminology

Table 1: Terminology Table

Term/Abbreviation	Definition/Description
MACARON	Multi-cloud region Aware Cache Auto-configuRatiON.
AWS	Amazon Web Services. A subsidiary of Amazon providing on-demand cloud computing platforms and APIs to individuals, companies, and governments, on a metered pay-per-use basis.
Write-Back Cache	A caching strategy where modifications to data in the cache aren't immediately mirrored to the storage device. The modified data is written back only when it's evicted from the cache.
JIRA	A proprietary issue tracking product developed by Atlassian that allows bug tracking and agile project management.
LFU	A cache eviction algorithm that removes the least frequently accessed items first to make space for new data.
Clock Algorithm	A cache eviction policy designed to approximate the least recently used algorithm without requiring its extensive record-keeping.
Egress Costs	The fees associated with transferring data out of a cloud computing environment to another network or cloud service.

References

- [1] Hojin Park, Ziyue Qiu, Greg Ganger, and George Amvrosiadis. Macaron: Multi-cloud/region aware cache auto-reconfiguration. In *Unpublished Manuscript*, 2024.

Reflection

-

Changes To Previous Deliverables